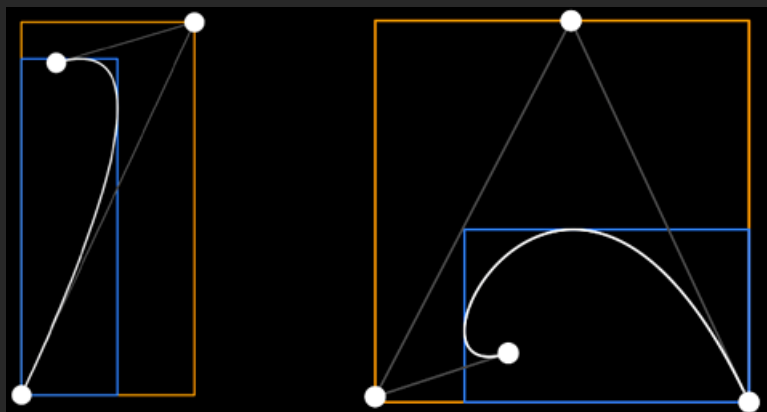


Inigo Quilez :: articles :: bezier bounding box - 2018

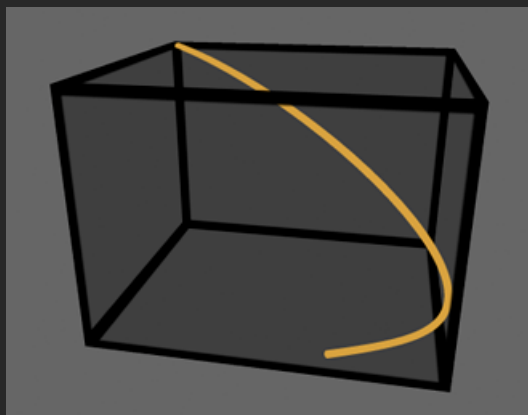


Intro

Bezier curves are useful primitives for computer graphics. From hair to font types, they show up almost everywhere. Having their bounding box computed quickly is important for keeping them organized spatially for fast query or evaluation. While simple approximations to the bbox are trivial (such as computing the bounding box of their control points), in this article we deduce the exact bounding box analytically.



A quadratic and a cubic Bezier segments, the bounding box of their control points in yellow, and their exact bounding box in blue



A 3D cubic Bezier and its bounding box

Quadratic Bezier

We start with a quadratic Bezier segment passing through points **p0** and **p2** and tangent control point **p1**, defined by

$$p(t) = (1-t)^2 \cdot p_0 + 2 \cdot (1-t) \cdot t \cdot p_1 + t^2 \cdot p_2$$

We know its bounding box is equal or smaller to the bounding box of the convex hull defined by its three control points **p0**, **p1** and **p2** - the whole curve must be contained within the convex hull of the control points, since the weights of each control point add up to exactly 1:

$$(1-t)^2 + 2 \cdot (1-t) \cdot t + t^2 = 1$$

We note that if **p1** happens to be inside the bounding box defined by **p0** and **p2**, we are done because that's the bounding box of the whole Bezier segment. If **p1** is outside of it, then the bounding box of the segment will be smaller than the bounding box of the three control points but bigger than the bounding box defined by **p0** and **p2**. In order to find the exact extent of it, all we need to do is compute the maxima/minima of the segment's coordinates at some point along the curve, which we can do by computing the curve's derivative

$$p'(t) = t \cdot (2p_0 - 4p_1 + 2p_2) + 2 \cdot (p_1 - p_0)$$

and equating it to 0, which results in

$$t = \frac{p_0 - p_1}{p_0 - 2p_1 + p_2}$$

where the division here is expressing a component-wise division. Meaning, for a 2D Bezier segment, we'll evaluate the above equation for the **x** and **y** components of **p0**, **p1** and **p2**, resulting in two values for **t**, which when plugged back in the formula for the Bezier segment **p(t)** will give us our min/max bbox extent.

The simplest and more direct code to perform the above could look like the following:

```

vec4 aabbBezier( in vec2 p0, in vec2 p1, in vec2 p2 )
{
    vec2 mi = min(p0,p2);
    vec2 ma = max(p0,p2);

    if( p1.x<mi.x || p1.x>ma.x || p1.y<mi.y || p1.y>ma.y )
    {
        vec2 t = clamp((p0-p1)/(p0-2.0*p1+p2),0.0,1.0);
        vec2 s = 1.0 - t;
        vec2 q = s*s*p0 + 2.0*s*t*p1 + t*t*p2;
        mi = min(mi,q);
        ma = max(ma,q);
    }

    return vec4( mi, ma );
}

```

If you want smaller and branchless code, the above can be encoded in the following form as well, which you'll also find in my [List of Bounding Boxes](#):

```

vec4 aabbBezier( in vec2 p0, in vec2 p1, in vec2 p2 )
{
    vec2 a = p0-2.0*p1+p2;
    vec2 b = p1-p0;
    vec2 t = clamp(-b/a,0.0,1.0);
    vec2 q = p0+t*(2.0*b+t*a);
    return vec4(min(min(p0,p2),q),
                max(max(p0,p2),q));
}

```

You can see the code above running in realtime here: <https://www.shadertoy.com/view/lsyfWc>. Also note that in order to make this work in 3D space, all you need to do is simply promote all data types above from **vec2** to **vec3**. This is an example: <https://www.shadertoy.com/view/tsBfRD>.

Cubic Bezier

The process to compute the bounding box of a cubic Bezier segment is analogous. The derivative becomes a second degree polynomial, which of course has also an analytic solution involving one square root. So, let's see:

$$p(t) = (1-t)^3 \cdot p_0 + 3 \cdot (1-t)^2 \cdot t \cdot p_1 + 3 \cdot (1-t) \cdot t^2 \cdot p_2 + t^3 \cdot p_3$$

Again the base functions follow the binomial coefficients, and add up to 1. The derivative of the curve is actual a second degree bezier curve itself, with tangents as control points:

$$p'(t)/3 = (1-t)^2 \cdot (p_1 - p_0) + 2 \cdot (1-t) \cdot t \cdot (p_2 - p_1) + t^2 \cdot (p_3 - p_2)$$

Note I divided the derivative by 3 to make it easier to recognize it as a second degree Bezier, since we don't care about constant factor anyways for our minima/maxima search. Speaking of what, equating the derivative with zero gives us a quadratic equation of the form

$$at^2 + 2bt + c = 0$$

with of course

$$t = \frac{-b \pm \sqrt{b^2 - ac}}{a}$$

where

$$a = -1 \cdot p_0 + 3 \cdot p_1 - 3 \cdot p_2 + 1 \cdot p_3$$

$$b = 1 \cdot p_0 - 2 \cdot p_1 + 1 \cdot p_2$$

$$c = -1 \cdot p_0 + 1 \cdot p_1$$

and the three coefficients are vectors really (for we are working in N dimensions, so the above quadratic are really N simultaneous quadratic equations to be solved). Also, these coefficients show yet again one more way the Bezier curves are related to Pascals's triangle. If we rewrite the above as a matrix multiplication it might be more obvious:

$$k = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 1 & -2 & 1 & 0 \\ -1 & 3 & -3 & 1 \end{pmatrix} \cdot \begin{pmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{pmatrix}$$

where **k** is a Nx2 matrix.

Now with all the quadratics in place, we are ready to solve the equation and compute our analytically perfect and tight bounding box (in this case, for a 2D curve)!

```
vec4 aabbBezier( vec2 p0, vec2 p1, vec2 p2, vec2 p3 )
{
    vec2 mi = min(p0,p3);
    vec2 ma = max(p0,p3);

    // d=position, c=velocity, b=acceleration, a=jerk
    // following finite differences with binomial/Pascal's coefficients
    vec2 d = p0;
    vec2 c = -1.0*p0 + 1.0*p1;
    vec2 b = 1.0*p0 - 2.0*p1 + 1.0*p2;
    vec2 a = -1.0*p0 + 3.0*p1 - 3.0*p2 + 1.0*p3;

    vec2 h = b*b - a*c;

    if( h.x > 0.0 )
    {
        h.x = sqrt(h.x);
        float t = (-b.x - h.x)/a.x;
        if( t>0.0 && t<1.0 )
        {
            float s = 1.0-t;
            float q = s*s*s*p0.x + 3.0*s*s*t*p1.x + 3.0*s*t*t*p2.x + t*t*t*p3.x;
            mi.x = min(mi.x,q);
            ma.x = max(ma.x,q);
        }
        t = (-b.x + h.x)/a.x;
        if( t>0.0 && t<1.0 )
        {
            float s = 1.0-t;
            float q = s*s*s*p0.x + 3.0*s*s*t*p1.x + 3.0*s*t*t*p2.x + t*t*t*p3.x;
            mi.x = min(mi.x,q);
            ma.x = max(ma.x,q);
        }
    }

    if( h.y>0.0 )
    {
        h.y = sqrt(h.y);
        float t = (-b.y - h.y)/a.y;
        if( t>0.0 && t<1.0 )
        {
            float s = 1.0-t;
            float q = s*s*s*p0.y + 3.0*s*s*t*p1.y + 3.0*s*t*t*p2.y + t*t*t*p3.y;
            mi.y = min(mi.y,q);
            ma.y = max(ma.y,q);
        }
    }
}
```

```

    }
    t = (-b.y + h.y)/a.y;
    if( t>0.0 && t<1.0 )
    {
        float s = 1.0-t;
        float q = s*s*s*p0.y + 3.0*s*s*t*p1.y + 3.0*s*t*t*p2.y + t*t*t*p3.y;
        mi.y = min(mi.y,q);
        ma.y = max(ma.y,q);
    }
}

return vec4( mi, ma );
}

```

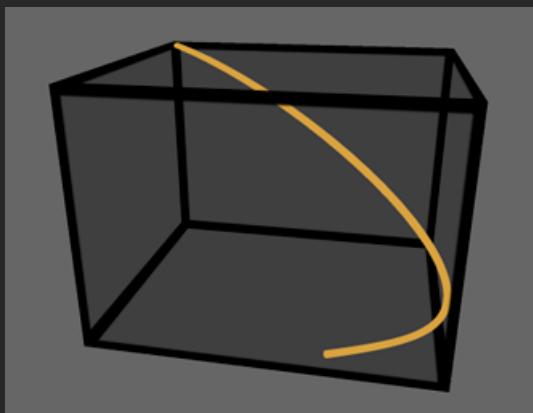
Or if a more compact and less branchy version is your thing, then you can encode the above like this instead:

```

vec4 aabbBezier( in vec2 p0, in vec2 p1, in vec2 p2, in vec2 p3 )
{
    vec2 c = -p0+    p1;
    vec2 b = p0-2.0*p1+    p2;
    vec2 a = -p0+3.0*p1-3.0*p2+p3;
    vec2 g = sqrt(max(b*b-a*c,0.0));
    vec2 t1 = clamp((-b-g)/a,0.0,1.0);
    vec2 t2 = clamp((-b+g)/a,0.0,1.0);
    vec2 q1 = p0+t1*(3.0*c+t1*(3.0*b+t1*a));
    vec2 q2 = p0+t2*(3.0*c+t2*(3.0*b+t2*a));
    return vec4(min(min(p0,p3),min(q1,q2)),
                max(max(p0,p3),max(q1,q2)));
}

```

Either way, you have a realtime running reference code here: <https://www.shadertoy.com/view/XdVBWd>. And again, extending the above code to 3D is trivial - you just need to replace all **vec2** variables to **vec3**, which you can find here: <https://www.shadertoy.com/view/MdKBWt>



inigo quilez - learning computer graphics since 1994